

Notes (Week 1 Wednesday)

These are the notes I wrote during the lecture.

Write down a formula that characterises what must be true for the programs to differ.

The programs differ iff one program does thing2 while the other does thing3.

Left program is doing thing2

$$\neg(l \leq m < h) \wedge m < l$$

Right program is doing thing3

$$\neg(l \leq m < h) \wedge m \geq h$$

Q: can both these expressions be true at the same time?

$$\begin{aligned} & \neg(l \leq m < h) \wedge m < l \wedge \neg(l \leq m < h) \wedge m \geq h \\ \equiv & \text{(associativity, commutativity)} \\ & \neg(l \leq m < h) \wedge \neg(l \leq m < h) \wedge m < l \wedge m \geq h \\ \equiv & \text{(idempotence of } \wedge \text{ (} P \wedge P \equiv P \text{))} \\ & \neg(l \leq m < h) \wedge m < l \wedge m \geq h \\ \equiv & \text{(fact of arithmetic)} \\ & \neg(l \leq m \wedge m < h) \wedge m < l \wedge m \geq h \\ \equiv & \text{(de Morgan: } \neg(P \wedge Q) \equiv \neg P \vee \neg Q \text{)} \\ & (\neg(l \leq m) \vee \neg(m < h)) \wedge m < l \wedge m \geq h \\ \equiv & \text{(arithmetic)} \\ & (l > m \vee m \geq h) \wedge (m < l \wedge m \geq h) \\ \equiv & \text{(distribution } (P \vee Q) \wedge R \equiv (P \wedge R) \vee (Q \wedge R) \text{)} \\ & (l > m \wedge m < l \wedge m \geq h) \vee (m \geq h \wedge m < l \wedge m \geq h) \\ \equiv & \text{(idempotence + arithmetic)} \\ & (l > m \wedge m \geq h) \vee (l > m \wedge m \geq h) \\ \equiv & \text{(idempotence: } P \vee P \equiv P \text{)} \\ & l > m \wedge m \geq h \\ & \text{(transitivity of inequality)} \\ \Rightarrow & l > h \end{aligned}$$

Conclusion: the programs can differ! But only if $l > h$.

$n! = n \leftrightarrow (n = 1) \vee (n = 3)$
^ on its own, neither true nor false.
^ true in a state where n maps to 1
^ false in state where n maps to 2
...

A state is, formally,
a mathematical function from variable names to values.

Q: since state can change,
is time an implicit extra parameter?

A: no.

A logical formula is interpreted in exactly one state.

In programs, the state may change.

In logic, we'd then talk about different states:
the state before and after an operation for example

$A[x:=E]$ A, but with all *free* occurrences of x
replaced by E.

$$\begin{aligned} & \forall(x + 1 = 2 \wedge (\forall x. x+1=2)) \\ & \Rightarrow (E.60) \\ & (x + 1 = 2 \wedge (\forall x. x+1=2))[x:=1] \\ & = \\ & (1 + 1 = 2 \wedge (\forall x. x+1=2)) \end{aligned}$$

Q: is . multiplication?

A: No. it's just a separator
 $\forall x. P$ "for all x, P holds"
alternative notations:
 $\forall x \ P$
 $\forall x. \ P$
 $\forall x: \ P$

Q: difference between $\Rightarrow$ $\Rightarrow$?

A: Yes.

$\Rightarrow, \rightarrow$	Implication	a combinator on logical formulas
$\Rightarrow$	Entailment	a truth claim about the relation between two formulas

$P \Rightarrow Q$ in all states, if P holds, then Q must also hold.

```
//(y+1) even
x := y + 1
//x even
```

Scenario:

I dimly recall that in order to swap two variables,
I need to use a third variable as a temp, but I
can't quite remember how to get it right.

Convention here:

lower-case variables are program variables

X,Y are placeholders for constants.

think: X is the constant that holds
the original value of x

goal:

```
//y = Y  $\wedge$  x = X
z = y
//z = Y  $\wedge$  x = X
y = x
//z = Y  $\wedge$  y = X
x = z
//x = Y  $\wedge$  y = X
```

What did I do?

- Wrote a swap program, and proved it correct, at the same time
- I started from the result I wanted (postcondition), and filled in the program backwards, working to the initial state.

You can use logic not just check programs,
but to help write them.

Q: what does // mean

A: I'm pretending // is the comment syntax
of some programming language.

I'm annotating a program with logical formulas
saying what's true in the state at that particular
point of execution.

```
// precondition
prog
// postcondition
```

Read: if we execute prog in a state where precondition is true,
postcondition must be true after prog is done.

// both x,y are even, or both are odd

```
//(x+y) is even  $\vee$  (x is even  $\wedge$   $\neg$ x > y)
//((x+y) is even  $\vee$  x is even)  $\wedge$  (x+y is even  $\vee$   $\neg$ x > y)
// ((x+y) is even  $\wedge$  x > y)  $\vee$  x is even)
 $\wedge$ 
((x+y) is even  $\wedge$  x > y)  $\vee$   $\neg$ x>y
//((x+y) is even  $\wedge$  x > y)  $\vee$  (x is even  $\wedge$   $\neg$ (x>y))
if(x>y) then
  //((x+y) is even  $\wedge$  x > y)  $\vee$ 
  x is even  $\wedge$   $\neg$ (x>y)
   $\wedge$  x>y
  //(x+y) is even  $\wedge$  x>y
  //(x+y) is even
  x := x + y
  //x is even
else
  //((x+y) is even  $\wedge$  x > y)  $\vee$ 
  x is even  $\wedge$   $\neg$ (x>y)
   $\wedge$   $\neg$ (x > y)
  //x is even  $\wedge$   $\neg$ (x>y)
```

```

//x is even
y := x - y
//x is even
//x is even

```

Exponentiation

```

// True
// 1*B^E = B^E
x = 1
// x*B^E = B^E
e = E
// x*B^e = B^E
b = B
// x*b^e = B^E
while(e≠0)
  // x*b^e = B^E ∧ e≠0
  // x*b*b^(e-1) = B^E
  x = x*b
  // x*b^(e-1) = B^E
  e = e - 1;
  // x*b^e = B^E
// x*b^e = B^E ∧ e=0
// x*1 = B^E
// x = B^E

```

Q: We haven't actually proven termination though?

A: Nope! We proved that *if* the program terminates, it exponentiated correctly.
Typically, you divide these sorts of arguments in two bits:

- (partial) correctness: if we terminate, we do the right thing
 - invariant (something that doesn't change)
- termination: we terminate
 - variant (something that changes)

Specifically, we need to find some natural number-valued expression about the state that:

- decrease in value with every iteration
- when it's 0, the loop guard must be false.

Better exponentiation

```

x = 1
e = E
b = B
while(e≠0)
  //x*(b)^e = B^E ∧ e ≠ 0
  //x*(b)^e = B^E
  if(e even)
    //x*(b)^e = B^E ∧ e even

```

```
//x*(b^2)^(e/2) = B^E
e = e/2
//x*(b^2)^e = B^E
b = b*b
//x*b^e = B^E
else
  //x*b^e = B^E ∧ e odd
  x = x*b
  // ..
  e = e - 1
  //x*b^e = B^E
//x*b^e = B^E
// x*b^e = B^E ∧ e=0
// x = B^E
```